**React Hook Form + TypeScript**

**Setup & Core Typing**

- Defining your form shape as a type/interface first: type FormValues = { name: string; email: string; age: number }.
- useForm<FormValues>() — how the generic drives type-checking for every field you touch.
- register("fieldName") — how TS autocompletes and validates field names against FormValues (typo in field name = compile error).
- Typing handleSubmit: SubmitHandler<FormValues> for the success callback, SubmitErrorHandler<FormValues> for the error callback.

**Working with formState**

- Typing errors object — shape mirrors FormValues but each field is a FieldError | undefined.
- isSubmitting, isDirty, isValid — where these come from and their types.
- Accessing nested/array field errors safely (errors.address?.city?.message).

**Nested & Array Fields**

- Typing nested objects in FormValues (e.g. address: { street: string; city: string }) and how register("address.city") stays type-safe.
- useFieldArray<FormValues> — typing dynamic field arrays (e.g. a list of phone numbers), including fields, append, remove.
- Common pitfall: array field types needing an id field injected by RHF (FieldArrayWithId).

**Controlled Inputs with RHF**

- When you need Controller instead of register (custom components, third-party inputs like date pickers, MUI, etc.).
- Typing Controller<FormValues, "fieldName"> and its render prop's field/fieldState.
- Typing a reusable custom <Input /> wrapped in Controller — passing field props through with correct types.

**Default Values & Reset**

- Typing defaultValues — must satisfy Partial<FormValues> or full FormValues.
- Typing reset(values) and getValues(), setValue("field", value) — how TS enforces value types match the field.
- watch("field") typing — return type based on the field name generic.

**Validation Modes**

- Typing a custom resolver function signature manually (before adding Zod).
- Built-in validation rules (required, pattern, min, max) — these are mostly untyped strings/values, note the limits of type safety here (this is exactly why Zod matters — segue to next section).

**Submission Flow**

- Full typed example: <FormValues> → useForm → register/Controller → handleSubmit(onSubmit) → typed onSubmit: SubmitHandler<FormValues> that calls a typed API function.
- Typing async submission (loading/error state around the submit handler).

## Zod + TypeScript

**Core Concept**

- Zod as a runtime validator that also gives you compile-time types — the "define once" philosophy vs writing an interface separately.
- z.infer<typeof schema> — deriving a TS type directly from a Zod schema (no duplication).
- Why this matters: your form type and your validation rules can never drift out of sync.

**Building Schemas**

- Primitives: z.string(), z.number(), z.boolean(), z.date().
- String refinements: .min(), .max(), .email(), .url(), .regex(), custom .refine().
- Optional vs nullable: z.string().optional() vs z.string().nullable() vs .nullish() — and how each maps to the inferred TS type (string | undefined vs string | null).
- Objects: z.object({...}), nested objects, and how z.infer produces nested types automatically.
- Arrays: z.array(z.string()), tuples with z.tuple([...]).
- Enums: z.enum([...]) vs z.nativeEnum(SomeTSEnum) — matching against a discriminated union.
- Unions & discriminated unions: z.union([...]) and z.discriminatedUnion("type", [...]) mapping cleanly onto TS discriminated unions from Day 2.

**Cross-Field & Custom Validation**

- .refine() for single-field custom rules (e.g. password strength).
- .superRefine() / schema-level .refine() for cross-field checks (e.g. password === confirmPassword), and typing the error path (path: ["confirmPassword"]).
- Transform vs refine: .transform() changes the output type, .refine() doesn't.

**Zod + React Hook Form Integration**

- @hookform/resolvers/zod — the zodResolver(schema) bridge.
- Wiring it: useForm<z.infer<typeof schema>>({ resolver: zodResolver(schema) }) — the single source of truth pattern.
- How RHF's errors object gets populated from Zod's validation issues automatically.
- Handling .transform() output types differing from input types (input schema vs output schema mismatch — a real gotcha with zodResolver).

**Schema Composition & Reuse**

- .extend(), .merge(), .pick(), .omit() on Zod schemas — mirroring TS utility types but at runtime too.
- .partial() for optional-everything variants (e.g. an "update" schema vs a "create" schema).
- Sharing one schema between frontend form validation and backend API validation (the big payoff of Zod).

**Beyond Forms**

- Validating API responses at runtime with schema.parse() / schema.safeParse() — typing the safeParse result ({ success: true; data: T } | { success: false; error: ZodError }).
- Environment variable validation with Zod (z.object({ API_URL: z.string().url() }).parse(process.env)) as a common real-world use case outside forms.

**Wrap-up**

- Mini exercise: build a signup form — Zod schema with email, password, confirmPassword (cross-field refine), age (number with min), wired through zodResolver into useForm, with Controller-wrapped custom inputs and fully typed onSubmit.